

E14 — Randomized SVD: Precision-Efficiency Tradeoff## Experiment

Overview This experiment investigates the **precision-efficiency tradeoff** of using Randomized SVD (RandSVD) as a substitute for full SVD within the Muon optimizer. Randomized SVD approximates the singular value decomposition using random projections, reducing complexity from $O(d^3)$ to $O(d^2 * (r+p) * q)$, where:-
p: Oversampling parameter (extra random vectors)- **q**: Power iterations (improves accuracy)
Key Question: Can RandSVD with appropriate (p, q) parameters match the convergence quality of exact SVD while reducing computational cost?
Tested Configurations:- Muon-Exact (baseline, full SVD)- Muon-RandSVD with p in {5, 10, 20} and q in {1, 2}- 10 random seeds per configuration

Scientific Question
Hypothesis: Increasing p (oversampling) and q (power iterations) improves RandSVD accuracy, but with diminishing returns. There exists an optimal (p, q) configuration that achieves near-exact convergence behavior at significantly reduced computational cost.
Null Hypothesis (H₀): RandSVD configurations do not differ significantly from exact SVD in convergence behavior.
Key Metrics:- K_epsilon: Iterations to convergence- min_loss: Best loss

achieved- time_s: Wall-clock time- I_conv:
Convergence flag (binary)- F_eps: Total FLOPs

Experimental Design | Parameter | Value | |-----
---|-----| | Problem | Matrix Sensing (MS) | | Base
algorithm | Muon | | Exact variant | Muon-Exact
(full SVD) | | RandSVD variants | p in {5, 10, 20}, q
in {1, 2} | | Dimension d | 50 | | Target rank r | 5
| | Learning rate | 0.01 | | Max iterations | 2000
| | Random seeds | 10 (0-9) | Total runs: 1 exact x
10 seeds + 6 (p,q) combos x 10 seeds = 70 runs

Data Loading & Inspection

```
In [ ]: import pandas as pdimport numpy as npimport matplotlib.pyplot as pltimport s
```

Exploratory Data Analysis### Summary by Configuration

```
In [ ]: # Summary for exactexact = df[df['algo'] == 'Muon-Exact']print('Muon-Exact:')
```

Comparative Analysis### Performance vs Parameters

```
In [ ]: # Aggregate summary tableconfigs = ['Exact'] + [f'RandSVD(p={p},q={q})' for
```

Visualizations

Plot 1: K_epsilon by RandSVD Parameters

```
In [ ]: fig, ax = plt.subplots(figsize=(14, 7))# Build data for grouped bar chartp_v
```

Plot 2: Time vs Accuracy Tradeoff

```
In [ ]: fig, ax = plt.subplots(figsize=(12, 8))# Plot each config as a pointexact_me
```

Plot 3: Convergence Rate Comparison

```
In [ ]: fig, ax = plt.subplots(figsize=(12, 7))# Compare K_epsilon distributionscont
```

Statistical Tests### Pairwise t-tests: Exact SVD vs RandSVD Configurations

```
In [ ]: from scipy.stats import ttest_relexact_k = df[df['algo'] == 'Muon-Exact']['k
```

Conclusions & Interpretation### Key Findings

- 1. RandSVD(p=20,q=2) achieves best convergence:** With K_epsilon ~39-40, it actually converges slightly faster than exact SVD on average, suggesting the approximation may have a regularizing effect.
- 2. Higher p and q improve results:** Configurations with larger oversampling (p=20) and more power iterations (q=2) consistently achieve lower K_epsilon and better minimum loss.
- 3. RandSVD(p=5,q=1) is the weakest:** This minimal configuration shows the highest K_epsilon (~45-47), demonstrating that insufficient approximation hurts convergence.
- 4. Time savings are modest at d=50:** All configurations have similar wall-clock times (~28-30s) since SVD overhead is small for d=50. Differences will amplify at larger dimensions.
- 5. No significant time difference from exact SVD:** At d=50, RandSVD does not provide statistically significant wall-clock savings compared to exact SVD (all p > 0.05 for time comparisons).

Recommendations-

- Best overall config:** RandSVD(p=20, q=2) offers the best convergence with competitive time.
- Best efficiency config:** RandSVD(p=10, q=2) provides a good balance.
- Avoid:** RandSVD(p=5, q=1) with minimal parameters shows degraded performance.
- Scaling insight:** At larger d (see E15), RandSVD's $O(d^2)$ vs $O(d^3)$ advantage will become significant.